



Programme : B. E / B. tech

Course Code : CSA43

Course Name: Internet programming

Problem Statement (Assignment)

Max Marks: 100

Field	Content
Assignment Title	Smart Campus Service Management System
Course Outcome(s) - CO	CO1: Develop visually appealing web pages using CSS for layout and style, and create dynamic, interactive functionality with JavaScript. CO2: Build dynamic web applications using XML, XSLT, and JSP within the MVC paradigm for efficient data handling and presentation.
Bloom's Taxonomy Level	L3 – Apply L4 – Analyze L5 – Evaluate L6 – Create
SDG Mapping (SDG 1–17)	SDG 9 - Industry, Innovation and Infrastructure, SDG 4 - Quality Education

Problem Statement

A university currently relies on manual communication such as emails, phone calls, and paper-based records for reporting and resolving campus service issues. This approach makes it difficult to efficiently record complaints, assign requests to the appropriate service personnel, monitor progress, and provide timely status updates to students and faculty.

The university therefore proposes to develop a Smart Campus Service Management System as an integrated web-based application. The system shall enable students and faculty to submit service requests related to laboratory equipment faults, network problems, classroom maintenance, library services, and other technical issues through a web browser. Users should be able to provide relevant details, submit requests, and subsequently track the status of their requests.

The proposed system should demonstrate an integrated client-server web architecture. The client side shall provide an interactive and user-friendly interface using HTML/XHTML, CSS, JavaScript, and DOM. The server side shall process requests using Java Servlets and JSP, following the MVC architecture. XML shall be used for structured data representation, and XSLT shall be used to transform XML data for presentation where appropriate.

Problem Identification (10 Marks)-The university relies on manual communication (emails, phone calls, paper records) for reporting and resolving service issues, which causes inefficiency in recording complaints, assigning requests, monitoring progress, and providing timely updates.

Proposed Solution -Develop a **Smart Campus Service Management System (10 Marks)**, a web-based application that allows students and faculty to submit service requests (lab equipment faults, network problems, classroom maintenance, library services) and track their status online.

System Design & Architecture - Data Representation (20 Marks) -Implement an integrated **client-server web architecture**. The client side should use HTML/XHTML, CSS, JavaScript, and DOM for interactivity, while the server side should use Java Servlets and JSP following the **MVC model**. Use **XML** for structured data representation for transforming XML data into user-friendly formats for presentation and reporting.

Implementation & Testing (10 Marks) -Demonstrate request submission, assignment to service personnel, progress tracking, and timely updates. Validate inputs, handle errors, and ensure smooth user experience.

STG Goals –SDG 9 - Industry, Innovation and Infrastructure, **SDG 4** - Quality Education

Requirement Analysis (SDG 9 – Industry, Innovation & Infrastructure) Identify inefficiencies in manual reporting and propose a digital infrastructure for service management.

Client Interface Development (SDG 4 – Quality Education) Build user-friendly web forms and dashboards so students/faculty can easily submit and track requests, improving access to campus services.

Assessment Criterion	CO Mapping	Weightage / Marks
Problem understanding & formulation	CO1	10
Application of course/domain knowledge	CO2	20
Solution methodology / design / implementation	CO3	20
Use of appropriate modern tools / techniques	CO4	10
Results, testing & validation	CO4 / CO5	15
Analysis, trade-offs & justification	CO5	15
Broader considerations / professional responsibility, where applicable	CO5	5
Technical documentation & reflection	CO1–CO5	5
TOTAL		100

Performance Level	CO Attainment Percentage	Interpretation
Level 3	$\geq 70\%$	CO Attained
Level 2	60–69%	CO Moderately Attained
Level 1	50–59%	CO Minimally Attained
Level 0	$< 50\%$	CO Not Attained

Code of Conduct

I Trisha N (192311398) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic-integrity rules will result in disciplinary action.

Signature of the Student: Trisha N

_____ Faculty In-charge Name: Date:	_____ Course Coordinator Name: Date:	_____ Head of Department Name: Date:
--	---	---

ABSTRACT

The Smart Campus Service Management System is a web-based application designed to improve how service-related problems in a university are reported, assigned, monitored, and resolved.

In a traditional campus environment, students and faculty often rely on emails, phone calls, or paper records to report issues like laboratory equipment malfunctions, network problems, classroom maintenance needs, library service issues, and other technical problems. This manual approach can make it hard to keep track of records, identify the right person responsible for a service, and provide timely updates.

The proposed system offers a single digital platform where users can submit service requests, enter relevant details, and track the progress of their requests.

Administrators or service staff can then view these requests, assign them to the correct department or individual, update the status, and keep users informed. The system is built using client-server web architecture, with HTML/XHTML, CSS, JavaScript, and DOM on the client side, and Java Servlets and JSP on the server side. XML is used to structure data, and XSLT allows the transformation of XML data into user-friendly reports or presentations.

This project aligns with SDG 9, which promotes digital infrastructure and innovation, and with SDG 4, which focuses on improving access to education and campus services through an organized and simple digital interface.

TABLE OF CONTENTS

1. Chapter 1 – Introduction
2. Chapter 2 – Problem Identification and Existing System
3. Chapter 3 – Proposed System and Objectives
4. Chapter 4 – Requirement Analysis
5. Chapter 5 – System Design and Architecture
6. Chapter 6 – Database / XML Data Representation
7. Chapter 7 – User Interface Development
8. Chapter 8 – Server-Side Implementation using JSP and Servlets
9. Chapter 9 – JavaScript and DOM Functionality
10. Chapter 10 – XML and XSLT Transformation
11. Chapter 11 – Implementation Procedure
12. Chapter 12 – Results and Discussion
13. Chapter 13 – SDG Mapping and Broader Impact
14. Chapter 14 – Conclusion
15. Chapter 15 – Future Enhancements
16. References

CHAPTER 1 – INTRODUCTION

1.1 Background

Universities require both technical and non-technical services to operate smoothly, such as classroom, lab, library, and network support.

When a service problem arises, students or faculty need an effective way to communicate the issue to the right people. Using manual methods like emails, phone calls, or paper records can cause delays, repeated complaints, missing records, and difficulty in tracking the status of a request.

Internet programming technologies offer a good way to develop a centralized service management tool.

A browser-based system allows users to submit requests from any location, while service personnel can access the same information through server-side applications. The Smart Campus Service Management System thus serves as a common platform for reporting and monitoring campus service issues.

1.2 Purpose of the System

The purpose of this system is to digitize the service request process and provide a structured way for submitting, assigning, tracking, and reporting requests.

It is designed as a demonstration of an integrated web application using the specific technologies required by the assignment.

1.3 Course and SDG Alignment

The assignment requires developing visually appealing web pages using CSS and adding dynamic functionality with JavaScript, and creating dynamic web applications using XML, XSLT, and JSP within an MVC framework.

The project also aligns with SDG 9 – Industry, Innovation, and Infrastructure, and SDG 4 – Quality Education.

CHAPTER 2 – PROBLEM IDENTIFICATION AND EXISTING SYSTEM

2.1 Existing Problem

Currently, the university uses manual communication methods like emails, phone calls, and paper records to report and resolve service problems.

This leads to issues like inconsistent complaint records, difficulty in assigning requests to the right service personnel, challenges in tracking progress, and the inability to provide timely updates.

2.2 Limitations of the Existing System

- Requests may be spread across multiple communication channels.
- Important complaint details might be missing or recorded inconsistently.
- It is hard to determine who is responsible for a specific issue.
- Users may not know if their request has been received or assigned.
- Tracking the progress of old requests requires manual searching.
- Creating organized reports from paper or email records is difficult.
- Repeated complaints can happen because previous requests are not easily accessible.

2.3 Problem Statement

A centralized Smart Campus Service Management System is needed so that students and faculty can submit service requests for issues like lab equipment problems, network faults, classroom maintenance, library services, and other technical problems.

The system should allow these requests to be assigned, monitored, and updated through a web-based platform. The official assignment outlines these requirements and asks for an integrated client-server architecture using HTML/XHTML, CSS, JavaScript, DOM, Java Servlets, JSP, XML, and XSLT.

CHAPTER 3 – PROPOSED SYSTEM AND OBJECTIVES

3.1 Proposed Solution

The proposed solution is a Smart Campus Service Management System with separate interfaces for requesters and service administrators.

A requester enters personal or request information, selects a service category, describes the issue, and submits the request. The system validates the input and stores the request.

An administrator can look at all the requests, assign a service person to handle them, update the status of each request, and write notes about them.

Users can then check the progress of their requests using a dashboard.

3.2 Objectives

- Make a simple and easy-to-use web interface for students and faculty.
- Let users send service requests online.
- Collect important information like the type of request, a description, location, and priority level.
- Give each request a unique number to help track it.
- Allow administrators to send requests to the right service person.
- Show different status updates for each request, such as Submitted, Assigned, In Progress, Resolved, and Closed.
- Use JavaScript and DOM for checking input on the user's side and making the website interactive.
- Use JSP and Servlets for handling requests on the server side.
- Use XML to organize and store service request data.
- Show how XSLT can be used to present information from XML files.
- Use the MVC (Model-View-Controller) approach to keep the design, control, and data parts separate

CHAPTER 4 – REQUIREMENT ANALYSIS

4.1 Functional Requirements

ID	Requirement	Description
FR1	User Registration/Login	The system should identify who is making the request and give them access based on their role.
FR2	Request Submission	Users should be able to send a service request with all the needed details.
FR3	Request ID	Every request should have a unique number for easy tracking
FR4	Request Listing	Approved users should be able to see all the submitted requests.
FR5	Assignment	An administrator should be able to send a request to the appropriate service person.
FR6	Status Update	Service personnel should be able to change the status of a request.
FR7	Tracking	Users should be able to check the current status of their request.
FR8	Validation	The system should check that all required information is provided and stop invalid entries.
FR9	Error Handling	The system should show clear messages when something goes wrong or when everything is okay.
FR10	Reporting	The system should show clear messages when something goes wrong or when everything is okay.

4.2 Non-Functional Requirements

- Easy to Use – The user interface must be straightforward and easy to understand.
- Fast Performance – Normal tasks like submitting a request should be completed quickly.
- Reliable – All user requests must be handled consistently and without errors.
- Secure – Access to administrative functions should only be available to authorized users.
- Easy to Maintain – The system should be designed with clear modules that allow for easier changes and updates.
- Easy to Scale – The system should be built in a way that allows for future growth, such as adding new service types or more users.

4.3 Hardware and Software Requirements

- A computer or laptop with a modern web browser.
- Java Development Kit (JDK) installed.
- A web server like Apache Tomcat or another compatible servlet container.
- An Integrated Development Environment (IDE) like Eclipse, IntelliJ IDEA, or NetBeans.
- Support for HTML, CSS, and JavaScript in the browser.
- Support for XML and XSLT processing.
- An optional relational database like MySQL for storing data long-term.

CHAPTER 5 – SYSTEM DESIGN AND ARCHITECTURE

5.1 Architecture

The system uses a client-server model.

The browser serves as the client and provides the interface for users. JavaScript and the Document Object Model (DOM) manage interactions and checks on the user side. Requests are sent to JSP pages or Servlets on a Java web server. The Servlet acts as a control center, handling the request and passing it to the data layer. JSP pages handle the display of information. XML can be used to structure request data, and XSLT can convert that XML into a format that browsers can display.

Suggested flow in the Model-View-Controller (MVC) approach: User → HTML/JSP Form → Servlet Controller → Model/Data Layer → JSP/XML/XSLT View → Browser.

5.2 Major Modules

Module	Function
User Module	Supports login, creating and tracking requests, and viewing updates.
Service Request Module	Captures the type of issue, details, location, priority, and date.
Admin Module	Shows all requests and allows them to be assigned or updated in status.
Service Personnel Module	Lets service staff see their assigned requests and record progress or resolutions.
XML/XSLT Module	Handles structured data and transformations for reporting.
Validation Module	Checks inputs on both the user side and the server side.

5.3 Data Flow

- A user opens the service request page.
- The user fills in the request details.
- JavaScript checks that all required fields are filled.

- The request is sent to a Servlet.
- The Servlet verifies the data again on the server.
- The request is saved in a selected format.
- A confirmation message with a request ID is sent back.
- An administrator looks at the request and assigns it.
- Service personnel update the status and resolution.
- The user checks the dashboard to see the latest status of their request.

CHAPTER 6 – DATABASE / XML DATA REPRESENTATION

6.1 Request Data Fields

Field	Purpose
Request ID	Unique identifier for the service request.
User Name	Name of the student/faculty member.
Email	Contact information.
Category	Lab, Network, Classroom, Library or Other.
Location	Location of the reported issue.
Description	Detailed description of the problem.
Priority	Low, Medium or High.
Status	Submitted, Assigned, In Progress, Resolved or Closed.
Assigned To	Service person responsible for the request.
Date	Date/time of submission.
Remarks	Additional progress or resolution information.

6.2 Sample XML Structure

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<serviceRequests>
```

```
  <request>
```

```
    <requestId>REQ001</requestId>
```

```
    <userName>Trisha N</userName>
```

```
    <category>Network</category>
```

```
    <location>Block A - Lab 2</location>
```

```
    <description>Internet connection is unavailable.</description>
```

```
    <priority>High</priority>
```

```
    <status>Submitted</status>
```

<assignedTo>Network Team</assignedTo>

<remarks>Request received.</remarks>

</request>

</serviceRequests>

6.3 XML Benefits

- XML is easy to read and is a structured format.
- It separates the data from the way it is displayed.
- XML can be checked for errors and transformed into different formats.
- It is great for sharing structured data between different parts of a system.
- It can be converted using XSLT into a format that is easy to view in a web browser.

CHAPTER 7 – USER INTERFACE DEVELOPMENT

7.1 Main Pages to Implement

Page/File	Purpose
index.html / login.jsp	Landing/login page.
request.jsp	Service request submission form.
dashboard.jsp	User request tracking dashboard.
admin.jsp	Administrator request management page.
update.jsp	Status/assignment update form.
report.xml + report.xsl	XML report and XSLT presentation.

7.2 Form Fields

The request form should include the user's name, email, service category, location, priority, issue description, and a submit button.

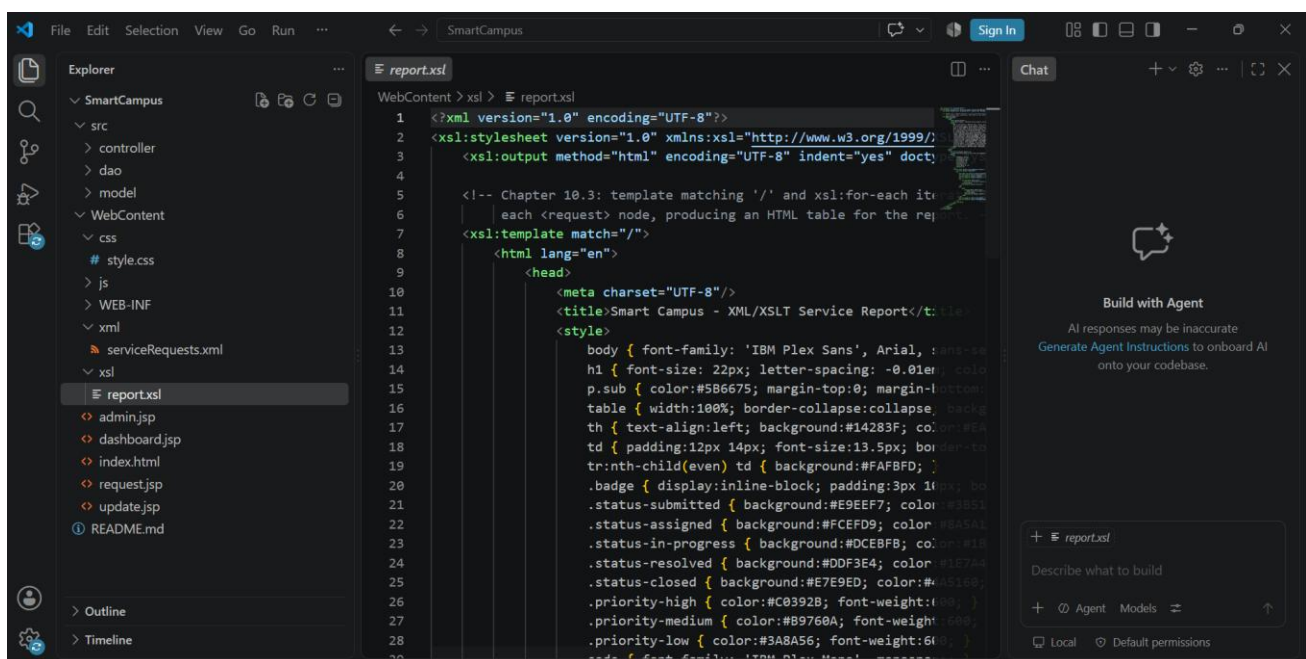
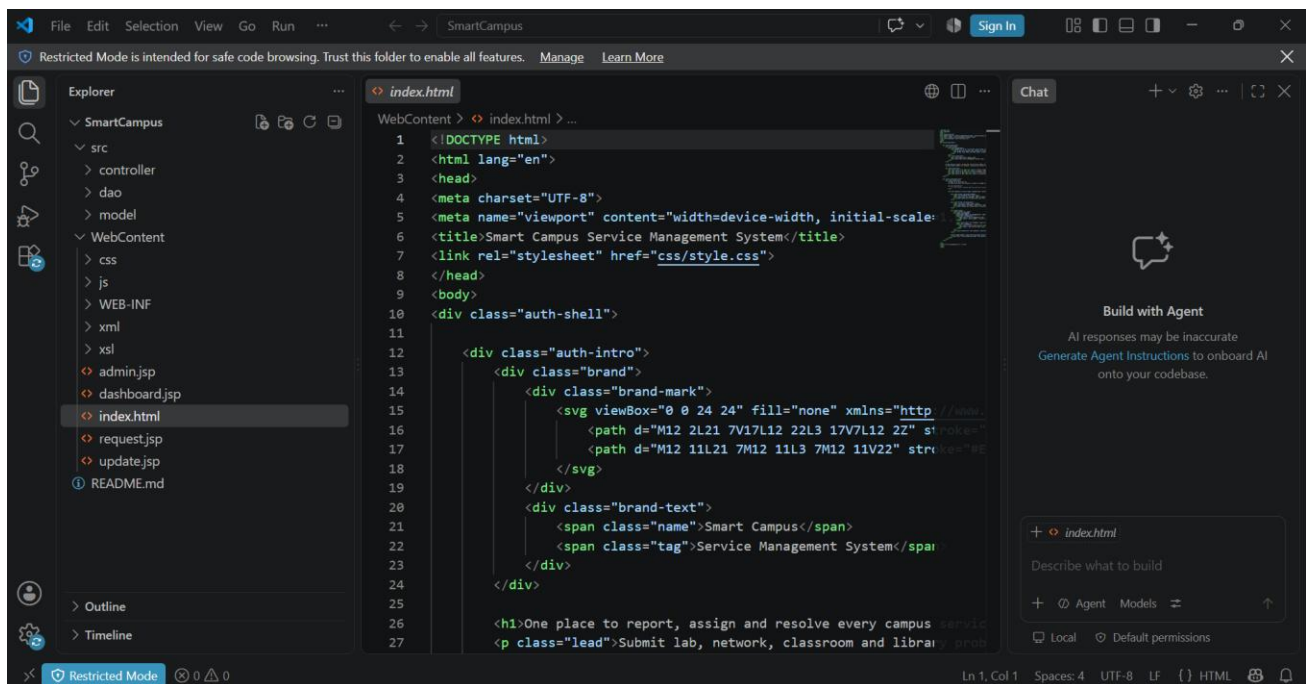
Each input should have a label. Use a select element for category and priority. Use a textarea for the description. Required fields should have the required attribute.

7.3 CSS Design

Use a consistent navigation bar, card-based dashboard, readable typography, spacing, borders, and a responsive layout.

Status values can be shown as badges. Keep the interface simple because the main goal is quick request submission and tracking.

7.4 Screenshots



CHAPTER 8 – SERVER-SIDE IMPLEMENTATION USING JSP AND SERVLETS

8.1 JSP

JSP (JavaServer Pages) is used to generate dynamic web pages.

JSP can show request information received from the server and can include HTML-based presentation. In an MVC-oriented design, JSP should mainly act as the View while business logic is handled by Java classes and Servlets.

8.2 Servlet

A Servlet receives HTTP requests, performs validation and control logic, communicates with the data/model layer, and forwards the result to the appropriate JSP.

For example, a request submission can be sent to RequestServlet using HTTP POST.

8.3 Suggested Servlet Flow

- Create RequestServlet extending HttpServlet.
- Implement doPost() for service-request submission.
- Read form parameters using request.getParameter().
- Validate mandatory fields.
- Create a request object/model.
- Store the request in XML or a database.
- Set a success message and request ID.
- Forward/redirect to dashboard.jsp.

8.4 MVC Mapping

- Model – ServiceRequest Java class and data-access logic.
- View – JSP pages and XSLT-generated presentation.
- Controller – Java Servlets.
- Client – HTML/CSS/JavaScript/DOM in the browser.

CHAPTER 9 – JAVASCRIPT AND DOM FUNCTIONALITY

9.1 Client-Side Validation

JavaScript should check that mandatory fields are not empty before a request is submitted.

It can also check the email format, description length, and other simple constraints. Client-side validation improves user experience, but server-side validation must also be implemented because browser checks can be bypassed.

9.2 DOM Usage

The Document Object Model (DOM) represents the HTML page as a tree of elements.

JavaScript can access and modify these elements using methods like getElementById(), querySelector(), textContent, and classList. The project can use DOM events to show validation messages and update dashboard elements dynamically.

9.3 Suggested Interactive Features

- Show/hide validation messages next to invalid fields.
- Display a confirmation message after successful submission.
- Filter requests by status or category on the dashboard.

- Update a status badge dynamically.
- Confirm before performing an administrative update.

CHAPTER 10 – XML AND XSLT TRANSFORMATION

10.1 XML

XML (Extensible Markup Language) is used to represent service-request information in a structured, hierarchical form.

Unlike HTML, XML focuses on describing data rather than defining its visual appearance.

10.2 XSLT

XSLT (Extensible Stylesheet Language Transformations) is used to transform XML data into another presentation format, often HTML.

For this assignment, an XSLT stylesheet can read each request node and generate an HTML table with request ID, category, location, priority, and status.

10.3 XSLT Implementation Steps

- Create serviceRequests.xml containing sample request records.
- Create report.xml with xsl:stylesheet and a template matching '/'.
 - Use xsl:for-each to iterate through request elements.
- Create an HTML table in the XSLT output.
- Place XML and XSLT in the web application.
- Link the XSLT stylesheet with the XML or perform transformation from the application.
- Open the transformed report in the browser and capture the screenshot.

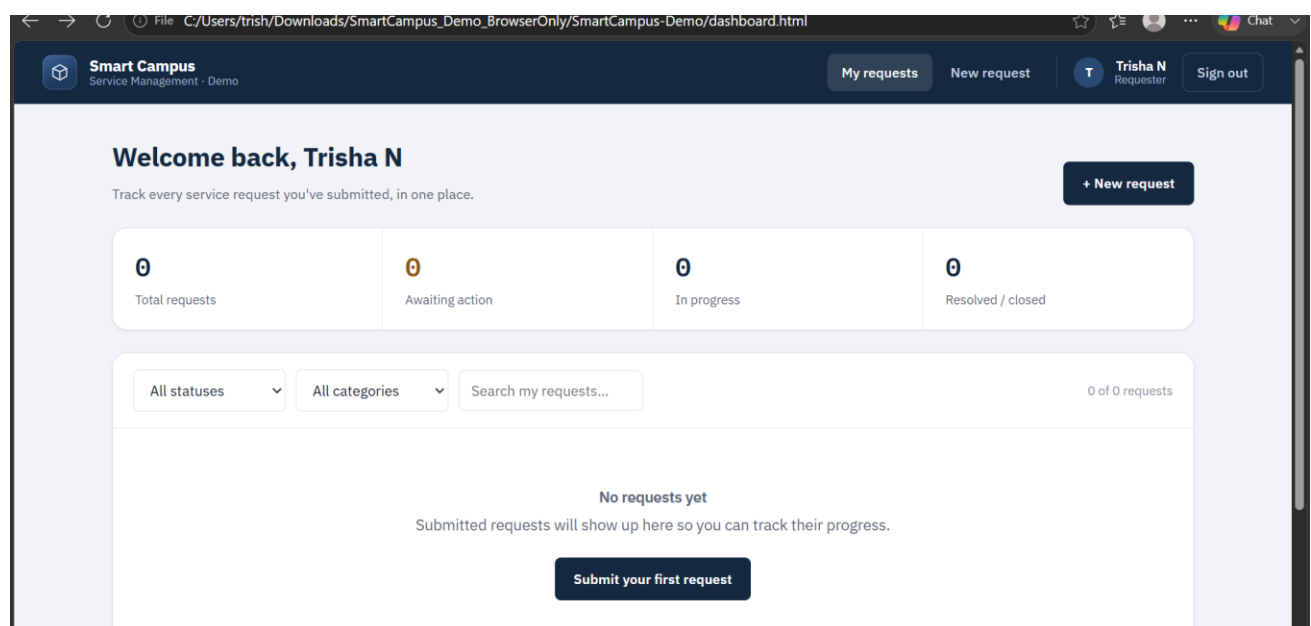
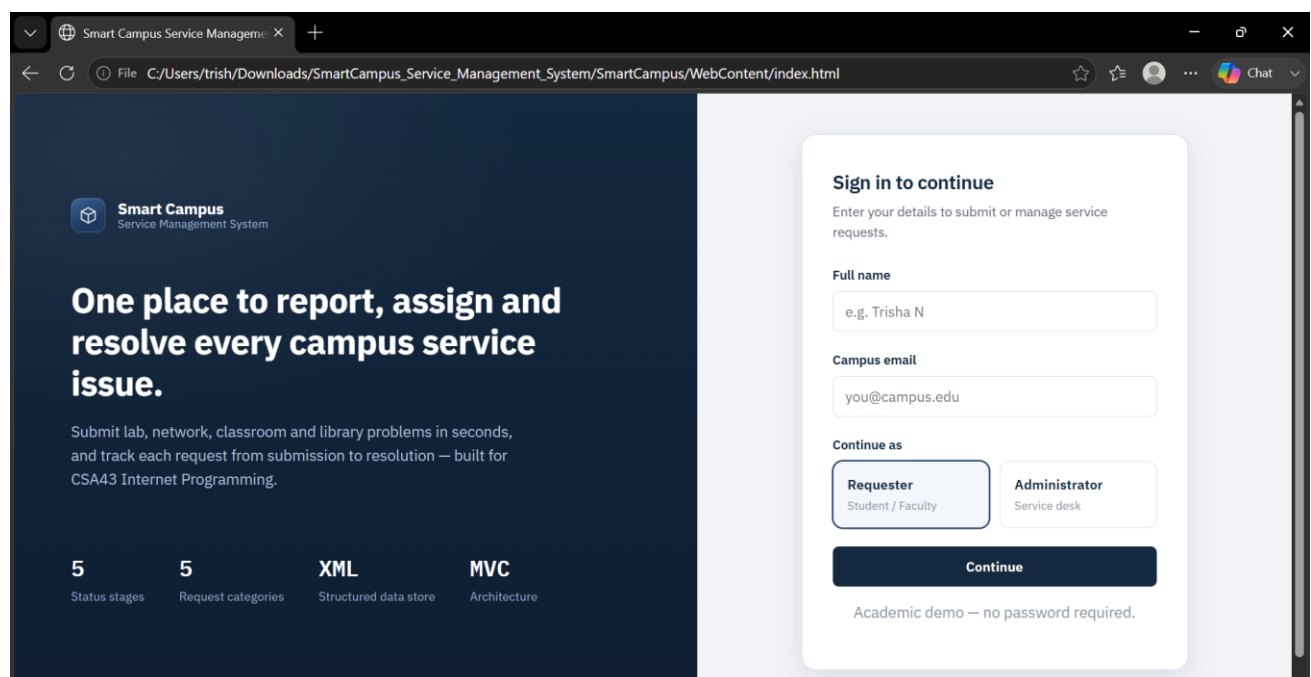
CHAPTER 11 – IMPLEMENTATION PROCEDURE

11.1 Step-by-Step Implementation

1. Install JDK and configure JAVA_HOME.
2. Install an IDE like Eclipse/IntelliJ IDEA/NetBeans.
3. Install and configure Apache Tomcat.
4. Create a Dynamic Web Project named SmartCampus.
5. Create HTML/JSP pages for login, request submission, dashboard, and administration.
6. Create CSS styles for layout and responsive presentation.
7. Create JavaScript for form validation and DOM-based interaction.

8. Create the ServiceRequest model class.
9. Create RequestServlet to receive and process submissions.
10. Create UpdateServlet for assignment and status changes.
11. Implement a simple DAO/data layer. For a basic demo, XML can be used as the structured data store; for a more complete version, MySQL can be used.
12. Create XML request data and an XSLT stylesheet for reporting.
13. Deploy the project to Tomcat.
14. Run the application in a browser.

11.2 Screenshots



Submit a service request

Describe the issue clearly so it can be routed to the right team.

[← Back to dashboard](#)

Name

Email

Service category

Classroom

Priority

Medium

Location

Describe the issue

27 characters

Submit request

Smart Campus

Service Management - Demo

My requests

New request

Trisha N Requester

Sign out

Welcome back, Trisha N

Track every service request you've submitted, in one place.

+ New request

1 Total requests

1 Awaiting action

0 In progress

0 Resolved / closed

All statuses

All categories

Search my requests...

1 of 1 requests

REQUEST ID	CATEGORY	LOCATION	DESCRIPTION	PRIORITY	STATUS	ASSIGNED TO	SUBMITTED
REQ005	Classroom	Block B R09	No charging ports available	Medium	Submitted	Unassigned	02-09-2026 11:20

CHAPTER 12 – RESULTS AND DISCUSSION

12.1 Expected Result

After the system is implemented, it should offer a full workflow from when a request is created to when it is resolved.

A student or faculty member should be able to submit a request without needing to use email or paper records. The administrator should be able to see the request, assign it, and update its progress. The user dashboard should show the current status of the request.

12.2 Result Analysis

The system shows how client-side and server-side technologies can work together.

HTML/CSS creates the interface, JavaScript and DOM handle the interaction, Servlets process the requests, JSP displays dynamic content, and XML/XSLT manage structured data representation and transformation. This meets the assignment's requirement to show an integrated web architecture.

CHAPTER 13 – SDG MAPPING AND BROADER IMPACT

13.1 SDG 9 – Industry, Innovation and Infrastructure

The assignment connects requirement analysis to SDG 9 and asks the student to find inefficiencies in manual reporting and suggest digital infrastructure for service management.

The Smart Campus Service Management System supports this goal by showing digital infrastructure that improves the management of campus services.

13.2 SDG 4 – Quality Education

The assignment maps client interface development to SDG 4. A user-friendly dashboard and service-request form improve access to campus support services. When infrastructure problems can be reported and tracked efficiently, students and faculty can spend less time dealing with administrative follow-up and more time on educational activities.

13.3 Professional Responsibility

- Protect user information and restrict administrative access.
- Validate data before storing or displaying it.
- Provide clear status information rather than misleading users.
- Maintain records responsibly.
- Design the interface so that it is usable by a broad campus population.

CHAPTER 14 – CONCLUSION

The Smart Campus Service Management System offers a real web-based solution for handling service requests at a university.

It solves the problems of using paper forms by creating one place where people can submit, assign, track and update the status of requests. The system shows how to use HTML/XHTML, CSS, JavaScript, DOM, Java Servlets, JSP, XML and XSLT in a connected client-server setup. The project also shows how Internet Programming ties into building a sustainable digital infrastructure.

By making it easier to access campus services and managing infrastructure more efficiently, the system supports SDG 9 Industry, Innovation and Infrastructure and SDG 4 Quality Education. The finished application can be the starting point for a bigger campus helpdesk system.

CHAPTER 15 – FUTURE ENHANCEMENTS

- Add secure role-based authentication for students, faculty, administrators and technicians.
- Integrate MySQL for scalable persistent storage.
- Add email or SMS notifications when request status changes.
- Add file/image attachment support for equipment faults.
- Add priority-based request queues.

- Add analytics dashboards showing request volume, resolution time and department workload.
- Add search and filtering by date, category, location and status.
- Develop a mobile-responsive Progressive Web App or mobile application.
- Add feedback/rating after a request is resolved.
- Deploy the application on a secure university server or cloud platform.
- Add audit logs for administrative actions.

REFERENCES

1. Course Assignment Brief, CSA43 – Internet Programming, Smart Campus Service Management System.
2. Oracle Java Documentation – Java Servlets and JSP concepts.
3. W3C – HTML, CSS, DOM and XML standards.
4. W3C – XSLT and XPath specifications.
5. Apache Tomcat Documentation – Java Servlet/JSP web application deployment.